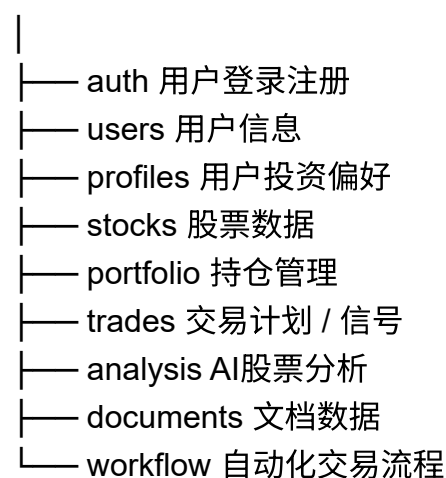


项目结构

API Router



整个系统在干什么

其实这是一个 **AI股票交易系统 backend**。

流程是：

用户注册



上传股票数据 / 文档



AI分析股票



生成交易计划



生成交易信号



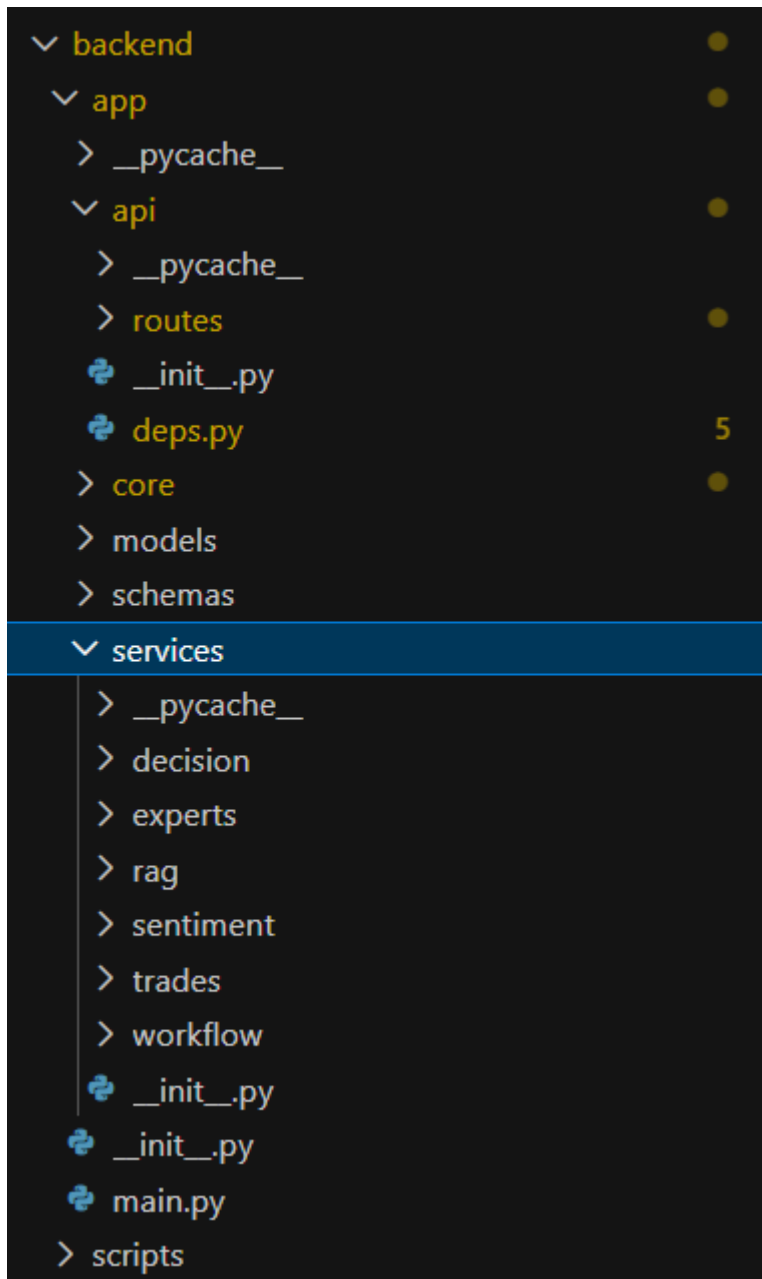
管理持仓

文件：trades

- `create_trade_plan(...)`：基于用户 + 股票生成一个完整“交易计划”
- `create_trade_signal(...)`：基于计划 + 当前价格生成“当前该怎么做”的信号

它把复杂逻辑放在 `app.services.trades.planner` 里，router 只负责：

- 校验权限
- 调 service
- 把 ORM 转成 response schema



1. api中的routes是前端页面路由
2. core是一些必要的服务，比如读取.env的配置文件、配置数据库、哈希密钥用户注册需要的功能
3. models中是一些数据库层面的东西，**股票分析/交易系统后端**的“数据层（ORM Models）+ 配置/数据库/安全”核心骨架：它们定义了数据库里有哪些表、表之间怎么关联、以及登录认证（JWT）和数据库连接怎么做
4. schemas是Pydantic的Schema（数据结构定义）：用来规定API的**请求体长什么样**、以及接口返回给前端的**响应长什么样**。它们通常放在 app/schemas/ 目录里。
5. services是系统里真正“会算”的那部分：**专家模块**
(**fundamental/financial/technical/news/macro**) + **文档检索（RAG）+ 情绪打分 + 信号融合引擎 + 交易计划/信号生成 + 每日扫描/复盘流水线**。它们把数据库里的行情、文档、用户偏好变成：**买/卖/观望 + 仓位 + 止损止盈 + 候选池/榜单**。
 - **retrieval/sentiment**：从数据库拿文档，用词典做情绪打分
 - **fundamental/financial/news/macro**：把文档情绪变成各自专家信号
 - **technical**：把行情数据变成技术面信号

- **engine**: 融合五个信号 + 用户偏好 → 输出 buy/sell/hold + 仓位 + 止损止盈 + 股数建议
 - **planner**: 把融合结果写成 TradePlan, 并能生成 TradeSignal
 - **pipeline**: 每天复盘生成候选池, 开盘前扫描生成榜单
 - ORM Model (你上一批) = 数据库表怎么存
-
- Router (你最开始那批) = API 怎么处理请求
 - **Schema** (你这批) = API 进来/出去的数据格式标准

1) core/config.py: 配置从哪来

Settings 会从 `.env` 和环境变量读取配置 (比如数据库地址、JWT 密钥、token 过期时间等)。全项目通过 `settings.xxx` 使用统一配置。

config

2) core/db.py: 数据库连接 & Session 注入

- `engine = create_engine(...)` 建立数据库连接引擎
 - `SessionLocal` 是“会话工厂”
 - `get_db()` 是 FastAPI 依赖: 每个请求创建一个 `db session`, 请求结束自动关闭, 避免泄漏。
-

3) core/security.py: 密码加密 & JWT token

- `get_password_hash()`: 注册时把明文密码变成 `bcrypt hash` 存库
 - `verify_password()`: 登录时验证密码
 - `create_access_token()`: 签发 JWT, 里面放 `sub=用户id` 和 `exp=过期时间`, 用 `settings.jwt_secret` 签名。
-

下面这些是 ORM Models: 每个类 = 一张表

4) user.py: 用户表

User 是系统的“主体”。字段:

- `email` 唯一
- `password_hash` 保存加密后的密码

- `created_at` 注册时间

并且它和其他表建立关系：`profile/analyses/positions/trade_plans/trade_signals`。

会被：注册登录（`auth`），获取当前用户（`users/me`），以及几乎所有需要登录的接口用到。

5) `profile.py`：用户投资偏好

`UserProfile` 是 `User` 的一对一扩展（`user_id` unique）：

- 风险偏好：`risk_level`, `max_drawdown`, `risk_budget`, `max_single_position`
- 投资风格：`investment_horizon`, `style`
- 财务信息：`income`, `assets`, `experience_years`
- `preferences`：JSON 可扩展偏好
它通过 `relationship` 绑定回 `User`。 `profile`

会被：`profiles.py` 读写；`analysis.py` / 交易策略用 `profile` 决策。

6) `stock.py`：股票基础信息

`Stock` 是股票主数据：

- `symbol` 唯一
- `name`, `market`, `sector`

它和很多表关联：`market_data`, `analyses`, `positions`, `trade_plans`, `trade_signals`。 `stock`

会被：`stocks/portfolio/trades/analysis` 等模块频繁引用。

7) `market.py`：行情 K 线数据

`MarketData` 保存每天行情（OHLCV）：

- `stock_id` 外键指向 `Stock`
 - `date`, `open/high/low/close`, `volume`
- `Stock` ↔ `MarketData` 是一对多。 `market`

会被：`analysis/trades` 在计算当前价格、技术指标、信号时用（你在 `analysis router` 里就取了最新 `close`）。

8) position.py: 用户持仓

Position 表示用户在某股票上的仓位：

- user_id、stock_id 外键
 - quantity 数量
 - avg_price 持仓均价
 - status open/closed
 - opened_at/updated_at/closed_at
- 还和 TradePlan 关联：一个持仓可以有多个交易计划。position

会被：portfolio 路由读写；analysis 融合决策时会读取当前仓位。

9) analysis.py: 一次“分析结果”记录

Analysis 是“用户对某股票做的一次 AI 分析结果”：

- user_id, stock_id
 - final_action (buy/sell/hold 之类)
 - position_size 建议仓位
 - rationale (JSON: 理由)
 - risk_notes (JSON: 风险点)
- 并且 Analysis 会挂很多 ExpertSignal (一对多)。analysis

会被：analysis 路由创建/查询，用来持久化每次分析。

10) expert_signal.py: 每个专家模块输出

ExpertSignal 是“某次分析里的某个专家模型的输出”：

- analysis_id 外键
 - expert_name (fundamental/technical/news...)
 - signal (buy/hold/sell)
 - score, confidence, horizon
 - key_factors, risk_flags, evidence 都是 JSON 列表
- 它属于某个 Analysis。expert_signal

会被：analysis 路由在保存分析时，逐个把专家信号写入数据库。

11) trade_plan.py: 交易计划

TradePlan 是“给用户的可执行交易计划（策略）”：

- 归属：user_id, stock_id, 可选 position_id
- side (long/short 或 buy/sell)
- entry_low/entry_high 进场区间
- ladder_prices 分批买入/卖出价格 (JSON list)
- stop_loss_price, take_profit_price, trailing_stop_pct
- suggested_shares 建议股数
- hold_days, valid_until
- status (active 等)
- rationale (JSON)

并且一个 plan 可以产生多个 TradeSignal。trade_plan

会被：trades 路由生成/列表。

12) trade_signal.py: 交易信号（更“实时”的动作建议）

TradeSignal 是“基于交易计划和当前价格生成的执行信号”：

- user_id, stock_id
- 可选 trade_plan_id
- side, signal_type (例如 entry/exit/reduce/add 等)
- trigger_price
- suggested_shares
- confidence
- reason (文本理由)
- created_at

属于 User/Stock/TradePlan。trade_signal

会被：trades 路由生成 signals、查询历史 signals。

13) document.py: 文档/资料库

Document 是“股票相关材料”存储：

- stock_symbol (注意这里不是外键，是字符串)
- doc_type (news/filing/report...)
- title, content, source
- published_at
- doc_metadata (映射到数据库列名 "metadata", JSON)

会被：documents 路由 ingest/list；也可能供新闻专家/基本面专家检索证据。

14) daily_recap.py: 每日复盘摘要

DailyRecap 是“按交易日保存的市场复盘”：

- trade_date (unique: 一天一条)
- market_summary , macro_summary
- top_movers (JSON list) daily_recap

会被：workflow 的 post-close-review 生成并复用（避免重复算）。

15) candidate_pool.py: 候选股票池

CandidatePool 是“某天筛选出来的候选股票及其评分”：

- trade_date , stock_symbol
- sentiment_score , data_score , total_score
- reasons , evidence (JSON lists) candidate_pool

会被：workflow 的 scan/复盘管线存结果、排序输出。

16) scan_result.py: 扫描结果（榜单）

ScanResult 是“某天扫描出来的排名结果”：

- scan_date , stock_symbol
- rank , score
- action (建议动作)
- notes (JSON) scan_result

会被：workflow 的 pre-open-scan 返回给前端，展示 Top N。

最后把关系串起来（这套系统的“数据流”）

- 登录：User (password_hash) + JWT (sub=user_id)
- 股票数据：Stock ← MarketData
- 用户交易：User + Stock → Position → TradePlan → TradeSignal

- AI 分析：User + Stock + Profile + MarketData → Analysis + ExpertSignal
- 自动化流程：DailyRecap / CandidatePool / ScanResult 存每天的扫描与复盘输出

4) 五个专家模块：各自算一个信号（signal/score/confidence）

这些模块都返回同一种结构：

```
{
  "expert_name": "...",
  "signal": "bullish|bearish|neutral",
  "score": 0~1,
  "confidence": 0~1,
  "horizon": "...",
  "key_factors": [...],
  "risk_flags": [...],
  "evidence": [...]
}
```

4.1 fundamental.py：基本面专家（靠文档情绪）

- 从文档库取：["company", "fundamental", "research"] 最近 365 天最多 5 篇
- 对这些文档做情绪打分，映射成 0~1 的 score
- 文档越多 confidence 越高；没文档就 low_coverage 风险。fundamental

4.2 financial.py：财务专家（靠财报/报告文本情绪）

- 取 ["financial", "earnings", "report"] 最近一年 5 篇
- 情绪转 score，没覆盖就 low_financial_coverage。financial

4.3 news.py：新闻专家（短期）

- 取 ["news", "announcement"] 最近 30 天 6 篇
- 情绪更敏感（sentiment / 8），并标记 headline_risk。news

4.4 macro.py：宏观专家（不绑定单只股票）

- stock_symbol=None，取 ["macro", "policy"] 最近 180 天 5 篇
- 算宏观情绪影响。macro

4.5 technical.py：技术面专家（靠行情数据算 SMA/momentum/volatility）

- 从 MarketData 取最近 60 根（日）数据
- 不足 20 条就低置信度 + low_liquidity_data technical
- 算：
 - SMA5 vs SMA20（趋势）

- 5 日动量 momentum
- 20 日波动 volatility
- 根据这些组合成 score；波动高就加 high_volatility 风险。 technical

项目运行流程

当你启动服务器：

```
uvicorn app.main:app --reload
```

系统会：

main.py

|



创建 FastAPI app

|



加载所有 API router

|



服务器启动

|



等待请求



举个真实请求例子

如果前端请求：

```
POST /api/v1/analysis
```

流程是：

main.py

|



analysis.router

|



analysis.py

|



AI专家模块

|



decision engine

|



返回 buy/sell/hold